

中国科学技术大学专业学位研究生学位申请成果

开题报告评审表说明

姓 名	邓杰	学 号	SA24225109	性 别	男
所在院系	软件学院		学科、专业	软件工程 085405	
学生类别			专业学位硕士		
拟撰写的学位申请成果题目			面向智能座舱的多模态实时数据引擎设计与实现		
开题院审日期			2026 年 1 月 23 日		
开题报告评审组成员名单					
姓名	职称	工作单位			
李曦	教授级高工	中国科学技术大学软件学院			
娄文启	特任副研究员	中国科学技术大学软件学院			
王斌武	特任副研究员	中国科学技术大学软件学院			
唐朝舜		中国科学技术大学软件学院			
评审小组意见：（是否通过开题论证，是否需要修改等） 缺技术难点分析及其解决方案					
☒ 通过			☐ 不通过		
学院盖章 2026 年 1 月 23 日					

中国科学技术大学

专业学位研究生学位申请成果开题报告

学位申请成果题目: 面向智能座舱的多模态实时数据引擎设计与实现

学生姓名: 邓杰

学生学号: SA24225109

校内导师: 王鹏焜

第二导师:

实践导师: 马骏

所在院系: 软件学院

专业类别: 电子信息 0854

专业领域: 软件工程 085405

填表日期: 2025/12/6

中国科学技术大学研究生院培养处

二零二五年三月制表

说 明

1. 学位申请成果的开题报告是保证研究生培养质量的一个重要环节，为了加强对研究生培养的过程管理，规范其学位申请成果的开题报告，特制此表。

2. 学位申请成果开题报告，应该在学位授予点或培养单位组织的学术报告会上报告，听取意见，论证后再填写此表。

3. 此表经导师签字后，交培养单位研究生教学管理办公室存档。

4. 研究生在提交学位申请时，必须提交该学位申请成果开题报告。

一、简况

研究生简况	姓名	邓杰	学号	SA24225109	性别	男
	培养方式	☒ 全日制 ☐ 非全日制				
☒ 专业学位硕士	☐ 专题研究类论文 ☐ 调研报告 ☐ 案例分析报告 ☐ 产品设计（作品创作） ☒ 方案设计 ☐ 其他					
☐ 专业学位博士	学位论文申请： ☐ 专题研究类论文 实践成果申请： ☐ 重大装备 ☐ 仪器设备 ☐ 其他硬件产品 ☐ 软件产品 ☐ 设计方案 ☐ 技术标准 ☐ 其他					
学位申请成果题目	中文	面向智能座舱的多模态实时数据引擎设计与实现				
	英文	Design and Implementation of a Multimodal Real-Time Data Engine for Intelligent Cockpits				
学位申请成果摘要和关键词	摘要	针对智能座舱场景下，上层智能应用（Agent）缺乏统一、高效的历史与实时数据获取通道，需直接面对底层碎片化接口及复杂时序组织的痛点，本课题拟设计并实现一种基于时间线的多模态资源引擎中间件，其核心在于构建面向服务的统一数据访问层，通过设计以时间轴为核心的抽象数据模型，屏蔽了底层异构传感器（如 CAN 信号、视频流）的数据组织差异。系统在内部实现了高效的时间线缓存与轻量级对齐机制，向上层提供标准化的时序数据查询接口。在系统架构上，采用分层解耦设计，结合共享内存零拷贝技术与异步并发模型，突破了大容量视频流与高频离散信号的写入性能瓶颈。针对嵌入式系统存储资源受限的特性，设计了基于策略模式的自适应分级存储机制，支持依据检查点与滑动窗口进行数据的智能精简与持久化同步。此外，系统内建了基于规则的主动触发器，实现了从数据被动查询到事件主动感知的模式转变。预期实验将在嵌入式计算平台上验证该引擎的性能，目标在百路信号并发接入下实现缓存查询响应时间低于 10ms，从而有效支撑上层智能体对复杂场景的理解与因果推理需求。				
		中文	智能座舱 多模态数据 时间线引擎 分级存储 零拷贝技术			
	关键词	英文	Intelligent cockpit/Multi-modal data/Timeline engine/Hierarchical storage/Zero-copy technology			

选题来源	博世企业课题
------	--------

二、选题依据

1. 阐述该选题的研究意义，或工程设计的价值和意义，国内外概况和发展趋势，选题的先进性和实用性，技术难度及工作量。

1.1. 选题的研究意义

本课题的研究意义集中体现在为车载智能系统构建稳健的数据基础设施、提升多模态资源管理的组织能力以及推动时间线驱动的数据管理范式在工程实践中的适用性三个方面。随着车辆感知系统的功能不断扩展，视觉图像、语音片段、环境传感器数据和用户交互事件在同一时刻并行产生，并呈现出格式多样、采样频率不一致、语义层次差异明显的特性^[1]。这种高度异构的数据形态使得传统依靠模块级缓存、缺乏统一时间戳机制、数据结构松散耦合的管理方式难以应对现代车载场景的需求。尤其在高并发读写、多模态同步与响应时间严格受限的环境中，这类传统架构的瓶颈被进一步放大。在本课题中提出的统一时间线模型及其对应的性能指标要求，正是当前工程体系在多模态资源管理方面遇到的实际困难的集中体现，也直接勾勒了本研究需要解决的技术边界。

从理论研究的视角看，依据统一时间线来组织多模态资源，使得不同数据模态之间的同步、关联与因果关系能够转化为具有明确结构的时间序列问题^[2]。这种方法在系统层面为跨模态一致性提供了形式化基础，也使得触发器执行机制、检索策略和推理逻辑更容易构建在精确定义的时序框架之上。通过引入时间戳对齐算法、分层索引结构、缓存一致性维护策略等关键技术，本课题能够为多模态资源管理研究提供系统化的分析框架，并为统一时间线理念在复杂系统中的可行性提供理论支撑。这一方向的研究不仅补足了现有文献中对于多模态资源统一管理缺乏统一抽象层的不足，也为探索跨模态关系的动态构建方式提供了新的思路^[3]。

此外，车载系统正在向深度感知、人机协同和多模态决策方向发展，系统内部数据量的增长速度远超传统软件架构的承载能力。从工程应用的角度而言，建设一个能够在严格时序约束下整合多模态资源、保证一致性并支持实时触发的资源管理引擎，对于车载 AI 系统的响应效率、整体稳定性以及软件平台的扩展性具有决定性作用。本课题提出的时间线驱动资源管理方法，能够在数据写入、检索和触发流程中降低延迟，改进系统在高负载条件下的可预测性，并提升平台对新模态接入或算法模块扩展的适配能力。这类能力不仅对单一车载产品具有基础性价值，也能够作为通用组件应用于更大范围的智能驾驶、智能座舱和多模态交互系统，为未来产业中

的数据管理架构提供可推广的技术路线。

综上几点，本课题兼具理论研究价值和工程实践意义。它在理论层面探索了多模态资源的时间序列组织机制，在工程层面为构建稳定、高效、可扩展的车载数据管理平台提出了切实可行的设计方案。随着车载智能系统的复杂度持续提高，本课题的研究成果有望为下一代多模态系统提供统一、可靠的数据基础结构，从而为相关技术的发展和落地提供长远支撑。

1.2. 国内外概况与发展趋势

随着汽车“新四化”（电动化、网联化、智能化、共享化）的深入发展，汽车电子电气架构（E/E Architecture）正经历从分布式向域集中式（Domain Centralized）乃至中央计算式（Vehicle Central Computer）的深刻变革。这一硬件架构的演进为软件定义汽车（SDV）奠定了物理基础，同时也对智能座舱的基础软件架构提出了全新的挑战与需求。

在国际范围内，以 Tesla、Bosch、Aptiv 为代表的领先企业已率先推动车载软件架构从面向信号（Signal-Oriented）向面向服务（Service-Oriented Architecture, SOA）转型。AUTOSAR Adaptive Platform 的发布标志着行业标准对高性能计算与动态服务配置的支持日益成熟。

当前，智能座舱软件架构普遍采用分层设计：底层为虚拟机监视器支持下的异构操作系统（如 QNX、Android、Linux）；中间层为负责通信与资源调度的中间件（Middleware）；上层为各类应用服务。然而，现有的中间件解决方案多侧重于跨核通信（IPC/RPC）与服务发现（Service Discovery），在多模态感知数据的统一资源管理方面仍存在不足。传统的 Android Automotive 或 QNX 系统往往将摄像头、麦克风、车辆信号视为独立的硬件抽象层（HAL）实例，缺乏统一的“资源池化”管理机制，导致上层应用在获取多源异构数据时面临接口繁杂、时序难以对齐等工程痛点^[4]。

随着人工智能（AI）与大模型（LLM）技术引入智能座舱，系统对环境感知数据的需求从单一的“瞬时状态读取”转变为“历史时序推理”^[5]。对于多模态数据处理方面，目前主流方案通常采用分散式的 Pipeline 处理视频流、音频流和 CAN 总线信号。这种离散处理方式导致数据在传输过程中丢失了统一的时间基准^[6]。当 AI Agent 需要结合 $T\{x\}$ 时刻的车辆信号与 $T\{x-n\}$ 时刻的驾驶员面部表情进行综合推理时，现有系统往往难以提供精确的时间对齐（Time Alignment）支持，导致多模态融合感知的准确性下降^[7]。

在安防监控与流媒体领域，基于环形缓冲区（Ring Buffer）的时间切片存储技术已相对成熟。但在嵌入式车载系统中，受限于内存资源（RAM）与存储介质（UFS）的读写寿命限制，如何在保证高并发写入（如多路摄像头高帧率录入）的同时，实现高效的时间线索引与持久化同步（Sync to Persistence），仍是当前工业界的技术难点。本课题提出的“基于时间线的多模态资源引擎”概念，正是为了解决这一问题，通过构建紧凑的内存布局与智能抽帧策略，在有限资源

下提供可回溯的历史数据服务^[8]。

从智能体（Agent）协同与资源调度发展趋势的角度来看，未来的智能座舱将不再是被动的指令执行者，而是具备主动服务能力的智能体（System Agent）。这就要求底层资源引擎具备主动触发（Trigger）与检查点（Checkpoint）机制^[9]。国外方面：学术界与工业界正在探索基于“黑板模式（Blackboard Pattern）”的改良架构，即通过一个中心化的资源管理器（Resource Manager）来维护系统当前的认知状态，各 AI 模块作为知识源订阅并更新状态。国内方面：随着各家高算力芯片（如高通 8295、地平线征程系列）的普及，国内研究开始聚焦于异构计算资源（CPU/GPU/NPU/DSP）的共享内存零拷贝传输技术。但在软件层面，如何设计一套通用的、支持插件化扩展（Plugins）的资源引擎，以屏蔽底层硬件差异并向上层 Agent 提供标准化的查询接口，仍处于探索阶段。

智能座舱基础软件技术演进的趋势是体系性且高度关联的。随着汽车电子电气（E/E）架构向域集中式和中央计算式的高度集成演进，传统的基于硬件驱动的软件封装模式已难以满足跨域协同与高阶智能应用的需求，因此，资源架构的服务化与标准化转型成为行业共识。通过将底层异构硬件抽象为语义化的“资源服务提供者”模式，系统利用如 DDS、 SOME/IP 或高性能本地 IPC 等标准通信协议，实现了资源的跨域透明访问和动态管理。此举旨在从根本上降低软硬件耦合度，并为系统的可扩展性奠定基础。然而，服务化带来的数据流复杂性，特别是针对智能化应用对场景理解与因果逻辑推理的依赖，对传统数据管理提出了挑战。这一挑战推动了第二个关键趋势，即多模态数据的时空一致性管理。新一代座舱架构强调为数据赋予精确的“时空标签”属性，通过构建基于时间线（Timeline-based）的存储与索引机制，实现了异构传感器数据在时间维度上的精准对齐，从而为上层 AI 模型提供具备严格时序逻辑和完整因果链的高质量上下文数据^[10]。此外，为了应对海量、高频多模态数据流带来的存储与性能压力，行业正加速采纳云边端协同的分级存储策略。该策略在本地利用高性能内存进行高频环形缓存以保障毫秒级的实时查询响应，同时结合数据价值密度与生命周期管理策略，将精简或关键数据异步同步至持久化存储介质或云端。这种精细化的分级存储架构旨在有限的车载计算与存储资源约束下，实现实时性、数据完整性与系统成本之间的最优平衡，是支撑智能座舱长期运行与功能升级的关键基础设施。

随着数据管理需求的提升，工业界尝试将云端成熟的时序数据库（如 InfluxDB, Prometheus）引入端侧，但在车载嵌入式计算平台（如高通 QC8397/CX1）上，通用 TSDB 方案在嵌入式车载环境下的资源适配性较差。通用 TSDB 专为处理数值型指标（Metrics）设计，采用 LSM-Tree 等结构优化高频写入，但极不擅长处理非结构化的二进制大对象（BLOB），如视频帧或音频片段。若强行将视频数据写入 TSDB，不仅会导致严重的写放大效应，加速 UFS 存储介质的老化，还

会因频繁的 **Compaction** 操作引发系统抖动,无法满足本系统对视频流数据进行“智能抽帧”和“检查点回溯”的精细化管理需求^[11, 12]。

主流 TSDB 多基于 Go 或 Java 语言开发,其运行时的垃圾回收 (GC) 机制会造成不可预测的延迟峰值^[13],这对要求 10ms 内完成查询响应的车载实时系统是不可接受的。此外,通用数据库庞大的守护进程与复杂的 HTTP/TCP 协议栈交互方式,在内存受限的嵌入式环境中显得过于臃肿,无法提供直接的函数级调用接口或内存直接访问能力。

本课题拟设计的“基于时间线的多模态资源引擎”,正是针对上述行业痛点与发展趋势,提出一种集成了多模态数据接入、时间线对齐缓存、条件触发机制及持久化管理的软件架构以支持如^[14]所述的大规模多模态数据。该架构旨在填补当前车载中间件在智能化历史数据回溯能力的空白,具有重要的工程应用价值与学术研究意义。

1.3. 选题的先进性和实用性

本课题的先进性体现于其对现有车载中间件功能边界的系统性突破,即通过核心机制的深度集成,解决多模态、高并发、历史数据回溯的难题。具体而言,本引擎通过将时间线缓存 (Timeline Cache) 确立为核心结构组件,在资源层实现了对所有异构多模态数据 (包括视频流、音频、车辆总线信号) 的时间戳对齐与索引。这种内建的时间一致性管理能力,从根本上消除了多传感器融合中的时序偏差问题,为上层智能体进行复杂场景理解和因果关系推理提供了精确的数据基础,从而超越了传统信号或事件驱动架构在处理长时序上下文数据方面的局限性^[15]。此外,本课题通过集成触发器 (Trigger) 功能,创新性地实现了从“被动查询”到“主动感知”的数据服务架构转变,允许智能体层注册数据状态转换规则并接收主动事件通知,这是支持座舱从“功能执行”向“主动服务”进化的关键软件支撑。最终,通过将底层硬件操作抽象为独立的资源提供者并向上层暴露统一的查询接口,本设计实现了高内聚、低耦合的模块化目标,体现了车载基础软件平台设计的前沿理念,实现了与底层异构计算平台和传感器的高度解耦。

另外本课题对工程应用具有迫切的现实价值与显著的效率提升潜力。作为上层 **Central System Agent** 框架运行的必要数据基座,引擎提供的高性能、时间对齐的历史数据查询能力,是保障高阶智能化应用 (如疲劳检测、环境感知、事件追溯) 实时性、准确性和鲁棒性的决定性因素。通过精细化的内存缓存与异步持久化机制,该策略有效平衡了数据实时性、存储成本和关键硬件 (如 UFS 介质) 寿命之间的多维矛盾,确保系统在高负载下仍能持续稳定地提供数据回溯服务。同时,本资源引擎作为一个中心化的资源管理平台,提供了标准化的注册、查询与格式化输出机制,极大地简化了应用层的开发流程和跨团队的数据接口对接成本。其模块化的架构设计也为系统未来功能的快速迭代和持续的 OTA 升级提供了坚实的软件基础设施支持,充分体现了其在加速产品开发周期与提升系统可维护性方面的实用价值。

1.4. 选题的技术难度和工作量

本选题依托于车载座舱智能化背景，旨在构建一套能够支撑大模型与智能体应用的高性能底层数据引擎。该系统需在嵌入式硬件资源受限的前提下，解决多模态数据的实时吞吐、时间序列对齐及持久化策略等核心问题，具有较高的技术门槛与饱满的工程工作量。

本课题的技术难点主要集中在异构数据的高效组织、查询响应的性能优化以及复杂的缓存同步策略设计三个方面：车载数据包含高频连续流（视频/音频）与稀疏离散信号（CAN），其数据特征差异巨大。技术难点在于如何设计一种通用的时间线数据模型（Unified Timeline Model），既能兼容不同格式的数据载荷，又能统一对上层暴露一致的访问语义（API）。本课题需在保证紧凑存储的前提下，设计高效的混合索引机制，支持 Agent 基于任意时间窗口进行快速回溯，并在查询时刻（Query-Time）动态完成数据的逻辑对齐，从而在查询灵活性与系统开销之间取得平衡，确保对于异构时序数据的统一抽象模型与高效检索架构。作为座舱底软系统的核心模块，引擎需满足严苛的非功能性需求。在资源数量为 5、信号数量不超过 100 的并发条件下，全类型资源缓存查询响应时间需控制在 10ms 以内，写入耗时需控制在 1ms 至 5ms 之间。这要求设计高效的缓冲区及共享内存机制，以减少内存拷贝带来的开销，同时需解决多智能体（System Agents, OEM Agents）并发访问下的数据竞争与同步互斥问题。受限于嵌入式系统的内存空间，引擎需实现从高速缓存（Cache）到持久化存储（UFS）的动态同步。技术难点在于设计可配置的持久化策略（Sync Policy），系统需根据预定规则或智能体触发的“检查点”（Check Point），对视频流等大数据量资源进行智能抽帧或关键帧提取，而非简单的全量转存^[16]。这涉及到复杂的流式数据处理与存储空间管理的平衡，以及在 Sync 过程中保证数据读写无阻塞的并发控制逻辑。

本课题工作量饱满，涵盖了从需求分析、架构设计到核心算法实现及验证的全过程：软件架构与接口设计方面，主要任务是完成基于时间线的多模态资源引擎的整体软件架构设计。这包括定义 Resource Provider（数据源）、Timeline Cache（核心缓存）、AI Resource Manager（资源管理）及上层 Agent 之间的交互。需详细设计不少于 5 种核心资源的标准化数据结构及查询接口，并支持输出模板的可配置化。核心模块的算法实现方面，包括基于 C++ 实现四大核心功能模块：Cache 模块：实现多模态数据的缓存写入与内存管理算法。Query 模块：实现支持时间范围、特定资源类型的高性能查询检索逻辑。Sync/Store 模块：实现基于检查点和阈值触发的缓存数据清洗、压缩及持久化同步逻辑。Trigger 模块：实现基于信号状态变更的主动触发通知机制。策略配置与性能验证的主要工作包含设计并实现灵活的策略配置文件解析机制，支持对缓存大小、持久化间隔、抽帧规则的动态配置。搭建模拟测试环境，模拟 Camera、Mic 及 Vehicle Signal 的数据输入流，对核心模块的写入与查询性能进行基准测试，验证系统在 10ms 响应级下的稳定

性与正确性。

2. 主要参考文献

- [1] GU J, LIND A, CHHETRI T R, et al. End-to-end multimodal sensor dataset collection framework for autonomous vehicles [J]. Sensors, 2023, 23(15): 6783.
- [2] SONG Z, PENG L, HU J, et al. Timealign: A multi-modal object detection method for time misalignment fusing in autonomous driving [J]. arXiv preprint arXiv:241210033, 2024.
- [3] HALLAC D, BHOOSHAN S, CHEN M, et al. Drive2vec: Multiscale state-space embedding of vehicular sensor data[C]//2018 21st International Conference on Intelligent Transportation Systems (ITSC). IEEE, 2018: 3233-3238.
- [4] FRIDMAN L, BROWN D E, ANGELL W, et al. Automated synchronization of driving data using vibration and steering events [J]. Pattern Recognition Letters, 2016, 75: 9-15.
- [5] WU Y, MA X, LI J, et al. Multi-modal sensor fusion-based deep neural network for end-to-end autonomous driving with scene understanding [J]. IEEE Sensors Journal, 2021, 21(10): 11781-11790.
- [6] XIA Z-X, LI Y, LIU Y, et al. Robust long-range perception against sensor misalignment in autonomous vehicles[C]//2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV). IEEE, 2025: 5761-5770.
- [7] LEVINSON J, THRUN S. Automatic online calibration of cameras and lasers[C]//Proceedings of Robotics: Science and Systems. RSS Foundation, 2013: 1-8.
- [8] GAL E, TOLEDO S. Algorithms and data structures for flash memories [J]. ACM Computing Surveys (CSUR), 2005, 37(2): 138-163.
- [9] LIMSOONTHRAKUL S, DAILEY M N, SRISUPUNDIT M, et al. A modular system architecture for autonomous robots based on blackboard and publish-subscribe mechanisms[C]//2008 IEEE International Conference on Robotics and Biomimetics. IEEE, 2009: 633-638.
- [10] WILSON B, QI W, AGARWAL T, et al. Argoverse 2: Next generation datasets for self-driving perception and forecasting [J]. arXiv preprint arXiv:230100493, 2023.
- [11] WON Y, JUNG J, CHOI G, et al. Barrier-enabled io stack for flash storage[C]//Proceedings of the 16th USENIX Conference on File and Storage Technologies (FAST 18). USENIX Association, 2018: 211-226.
- [12] BOUKHOBZA J, OLIVIER P, LIM W-K, et al. A survey on flash-memory storage systems: A host-side perspective [J]. ACM Transactions on Storage, 2025, 21(3): 1-59.
- [13] WU M, ZHAO Z, YANG Y, et al. Platinum: A cpu-efficient concurrent garbage collector for tail-reduction of interactive services[C]//2020 USENIX Annual Technical Conference (USENIX ATC 20).

USENIX Association, 2020: 159-172.

[14] ALIBEIGI M, LJUNGBERGH W, TONDERSKI A, et al. Zenseact open dataset: A large-scale and diverse multimodal dataset for autonomous driving[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). IEEE/CVF, 2023: 20178-20188.

[15] QIAO D, ZULKERNINE F, ANAND A. Cobefusion cooperative perception with lidar-camera bird's eye view fusion[C]//2024 International Conference on Digital Image Computing: Techniques and Applications (DICTA). IEEE, 2024: 389-396.

[16] LAKSHMINARAYANA N K, KC A, RAVI A. Large scale multimodal data capture, evaluation and maintenance framework for autonomous driving datasets[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops. IEEE, 2019: 4302-4309.

三、研究方法与技术路线

1. 课题内容

本课题旨在面向智能汽车座舱场景，设计并实现一套“基于时间线的多模态资源引擎”（Timeline based Multi-Modal Resource Engine）。该系统作为座舱软件架构的底层核心组件，致力于解决多模态感知数据（如视觉、语音、车辆信号等）的实时整合、高效缓存与按需分发问题，从而为上层 AI 代理（AI Agents）及大模型提供全场景理解所需的数据支撑。

本课题的具体研究内容主要包含以下五个方面：

(1) 系统的需求分析与总体架构设计

深入分析智能座舱对底层数据资源管理的非功能性需求（如低延迟查询、高并发写入）及功能性需求。基于分层架构思想，设计资源引擎的整体软件架构，明确定义资源管理器（Resource Manager）在系统中的核心地位及其与上层 System Agents、底层 Resource Provider 之间的交互边界。

(2) 基于时间线的多模态数据缓存机制研究

研究并设计核心组件“Timeline Cache”，实现对异构多模态数据（Camera 视频流、Mic 音频流、Vehicle Signal 车辆信号、Screen Capture 屏幕截图等）的统一接入与管理。重点解决多源异构数据在时间维度上的对齐与同步问题，确保所有资源数据均能以严谨的时间序列进行组织与存储。针对视频、图像等大容量数据，研究基于共享内存的高效管理与传输方案，以降低系统开销。

针对车载环境多模态数据（视频流、音频流、离散信号）在时间维度上的非对齐特性，本研究将设计一种基于红黑树（Red-Black Tree）与双向链表混合索引的时间线存储结构。

考虑到车载信号数据的高频写入与基于时间窗口的范围查询需求，采用红黑树作为一级索

引，以数据帧的时间戳（Timestamp）为键值，确保单个数据节点的查找、插入与删除操作维持在 $O(\log N)$ 的时间复杂度。同时，为了优化大范围连续时间数据的遍历性能（如回溯过去 10 秒的视频），在叶子节点间构建双向链表，将范围查询的时间复杂度优化为 $O(\log N+K)$ （ K 为窗口内元素数量），避免了递归遍历的开销。

针对不同模态数据载荷差异巨大的问题（如 CAN 信号仅数字节，而 4K 视频帧达数兆字节），设计分级内存池机制。对于高频小对象，采用固定块大小的内存池（Slab Allocation）进行管理，以消除频繁 malloc/free 带来的系统调用开销及内存碎片；对于大对象，则通过共享内存句柄引用，仅在 Timeline 节点中存储元数据指针，实现数据索引与实际存储的物理分离。

(3) 高效查询接口与事件触发机制设计

设计灵活且高效的“Query/Format”查询引擎，支持按时间点、时间段、语义标签（如“刹车时”）等多种维度的组合查询。研究并定义标准化的数据输出协议（如基于共享内存的 Zero-Copy 句柄或序列化 JSON），在数据输出层屏蔽底层不同传感器采样率不一致的物理细节。引擎内部将采用轻量级的逻辑对齐策略（如软时间窗），向上层返回逻辑上时间一致的数据切片，使得上层应用无需关注底层的同步算法逻辑即可直接使用数据。同时，研发 Trigger 触发器模块，构建基于状态机或规则引擎的事件监测机制。当底层数据满足预设的转换规则（如特定车辆信号变更）时，系统能够主动向 Agent 层发送事件通知，实现从“被动查询”到“主动服务”的能力跃升。

(4) 数据持久化同步与检查点机制研究

针对内存缓存容量受限的问题，设计 Sync 同步模块与 Store 存储模块，实现从高速缓存（Cache）到持久化存储（UFS）的分级存储策略。研究基于检查点（Checkpoint）的数据压缩与抽帧策略，支持智能体或触发器在关键事件时刻（如场景突变）设置标记点，仅保留关键时间窗内的高价值数据，从而在满足历史回溯需求的同时优化存储空间利用率。

(5) 核心算法实现与性能优化验证

基于嵌入式硬件平台（如 QC8397/CX1），完成核心模块的代码实现与工程落地。重点攻克多模态数据的高频写入与毫秒级查询响应技术难点，确保在复杂工况下（如 100+车辆信号并发）全类型资源查询响应时间不超过 10ms，单资源写入不超过 1ms。通过构建验证环境，对系统的稳定性、实时性及资源调度策略的有效性进行全面测试与评估。

2. 系统需求分析

2.1. 系统概述

该系统定位于智能座舱底层软件架构中的资源管理核心层，旨在构建一个连接底层资源提供者与上层消费者的高效数据枢纽。系统以时间线（Timeline）为核心抽象模型，将来源分散、频率不一、格式各异的多模态数据进行统一的时间对齐与封装（如图 1 所示）。在数据存储与

管理策略上，系统采用分级存储架构：一方面在内存中维护高性能的时间线缓存（Timeline Cache），以满足上层应用对毫秒级（<10ms）热点数据的快速查询需求；另一方面，通过可配置的持久化策略，将历史数据进行精简、抽样或基于检查点（Checkpoint）处理后同步至非易失性存储（UFS），以支持长周期的历史回溯与上下文推理。此外，该引擎不仅提供被动的数据查询服务（支持 Native API、RPC 及 SQL 等多接口），还内建了基于规则的主动触发机制。通过监控多模态数据的状态变化，引擎能够在预定义的业务规则被满足时，主动唤醒上层智能体进行推理决策。

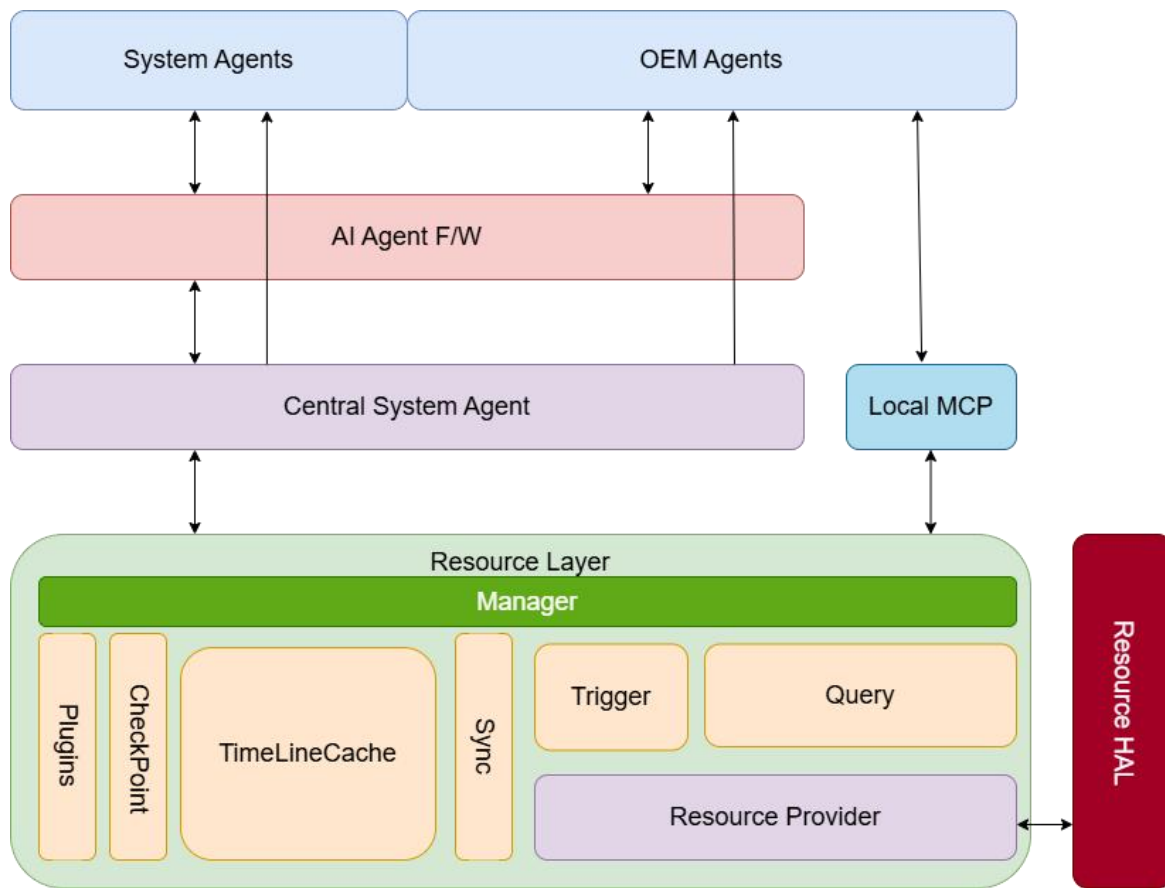


图 1 系统上下文与应用场景示意图

2.2. 功能需求

2.2.1. 支持多模态数据基于时间线的缓存

系统支持多模态资源数据缓存（如图 2 与图 3），包括但不限于：

Vehicle Signal

(1) Key-Value 格式数据。

(2) Groupable，不同信号有关联性，共同描述一组功能。

(3) Explainable, 支持插件解读为大模型可理解的文本描述。

Camera

(1) Multiple Camera Stream。

(2) 带时间戳的图像帧格式数据。

(3) 共享内存方式管理。

(4) System Agent 可能生成特定文本描述取代帧图像存储。

Audio

(1) Multiple Microphones stream。

(2) 带时间戳的 PCM 数据，长度由语音激活时长决定。可能由 Provider 截断。

(3) System Agent 可能生成特定文本取代语音存储。

Screen Capture

(1) Multiple Screens。

(2) BPM/PNG/JPEG 格式图像数据。

(3) 共享内存方式管理。

(4) System Agent 可能生成特定文本描述取代帧图像存储。

System Log

(1) Module - Log 格式数据。

(2) Explainable, 支持插件解读为大模型可理解的文本描述。

(3) System Agent 可能生成特定系统状态描述文本作为 log 存储。

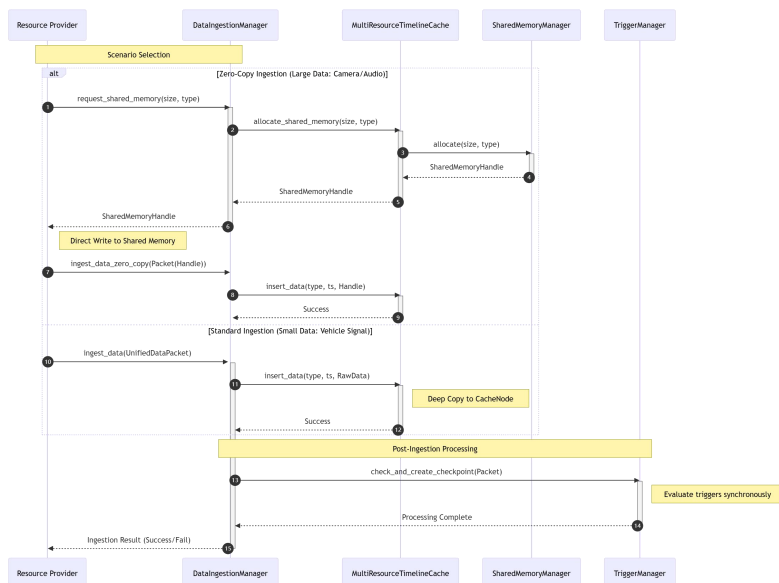


图 2 数据注入模块时序图

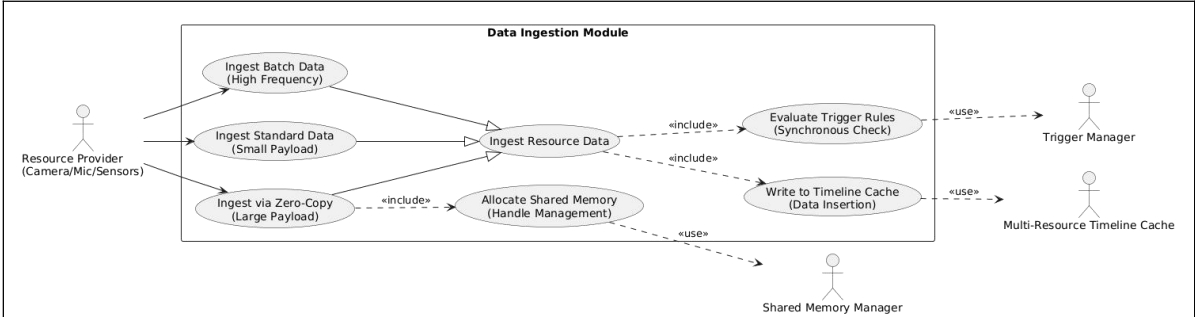


图 3 数据注入模块用例图

2.2.2. 支持多模态数据的查询，支持多种查询接口和格式化输出

在接口接入形式上，系统应提供 Native API、SQL 子集及 RPC/IPC 三种标准访问协议。

在核心检索功能上，查询引擎需具备多维度的时空组合检索能力：在资源维度，支持从全量资源集合、特定资源类型（如 **VehicleSignal**）深入至子资源粒度（如特定 **Camera** 通道或单一信号 ID）的精细化寻址；在时间维度，除支持基于闭区间的历史范围查询外，还需提供针对当前时刻或指定时刻的快照查询功能。鉴于离散时序数据的稀疏性特征，快照查询需内建可配置的回溯查找窗口机制（默认回溯 1 秒），以返回窗口内最新的有效状态数据，确保系统状态感知的连续性。

此外，查询结果的输出需支持基于模板的可配置格式化处理，通常采用 JSON 结构进行组织。针对视频流等大数据量资源，系统应仅返回共享内存句柄（Handle）以替代原始载荷，从而实现零拷贝的高效数据交付。同时，为弥合原始二进制信号与大模型语义理解之间的鸿沟，系统还需支持插件化的数据解释机制（**Explainable**），能够根据预定义规则将原始资源数值实时转译为具有语义信息的文本描述（如图 4 与图 5）。

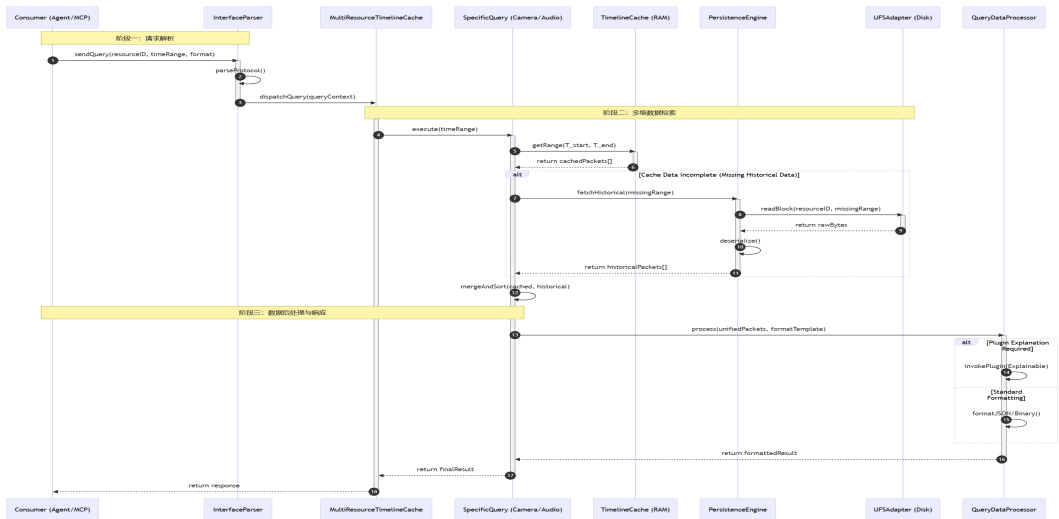


图 4 Query 模块时序图

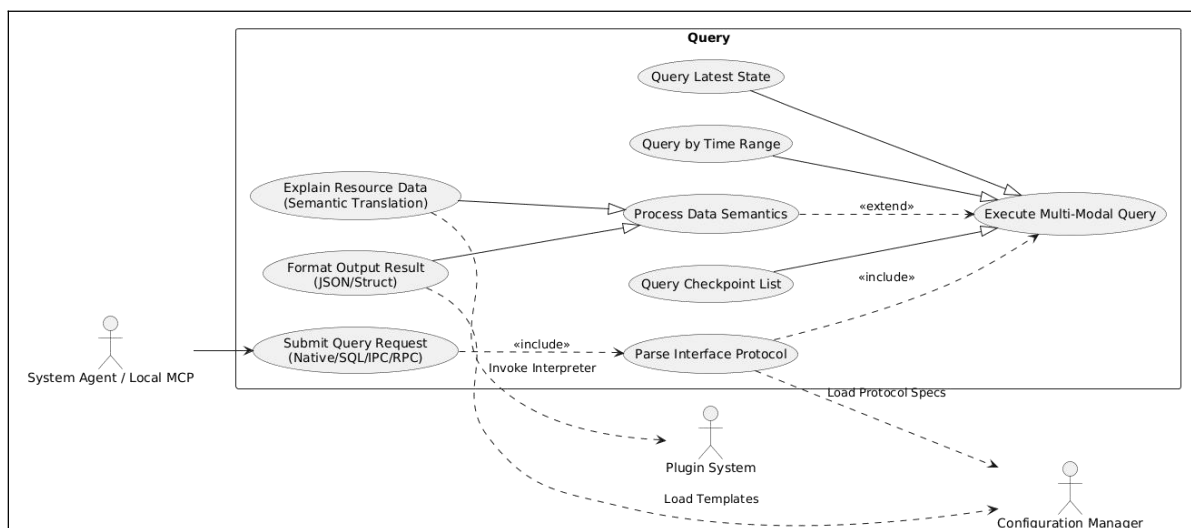


图 5 Query 模块用例图

2.2.3. 支持多模态数据按照预定规则和检查点时刻采样的方式精简存储

为解决多模态数据在有限内存资源下的长时效存储与高效访问矛盾，本系统将构建一套自适应的持久化存储与同步机制，通过灵活的策略编排实现从内存缓存到非易失性存储的智能数据交换（如图 6 与图 7）。在持久化策略配置方面，系统需支持针对异构资源的差异化降采样规则设定，例如针对高带宽的摄像头视频流配置抽帧间隔与数量，以实现存储空间的精细化管理。同时，系统应提供多维度的时空配额管理功能，允许对缓存区与持久化分区的容量上限进行动态配置，并支持设定后台持久化扫描的时间间隔，以及灵活启用或禁用检查点（Checkpoint）作为持久化触发的判定依据。

在数据同步执行层面，系统需建立高效的内存至存储介质的数据交换通道，支持多模态的触发逻辑以保障数据的一致性与完整性。具体而言，系统应具备基于容量阈值的被动交换能力，实时监控剩余缓存空间，一旦可用内存低于预设的安全水位线，即自动启动持久化同步以释放资源。此外，同步机制还需涵盖基于关键事件的主动触发与基于时间周期的定期刷新：当启用检查点策略时，系统将在生成关键时间标记时强制执行同步操作，确保重要场景数据的非易失性存储；而在常规状态下，则按照预设的时间步长周期性地数据写入存储设备。

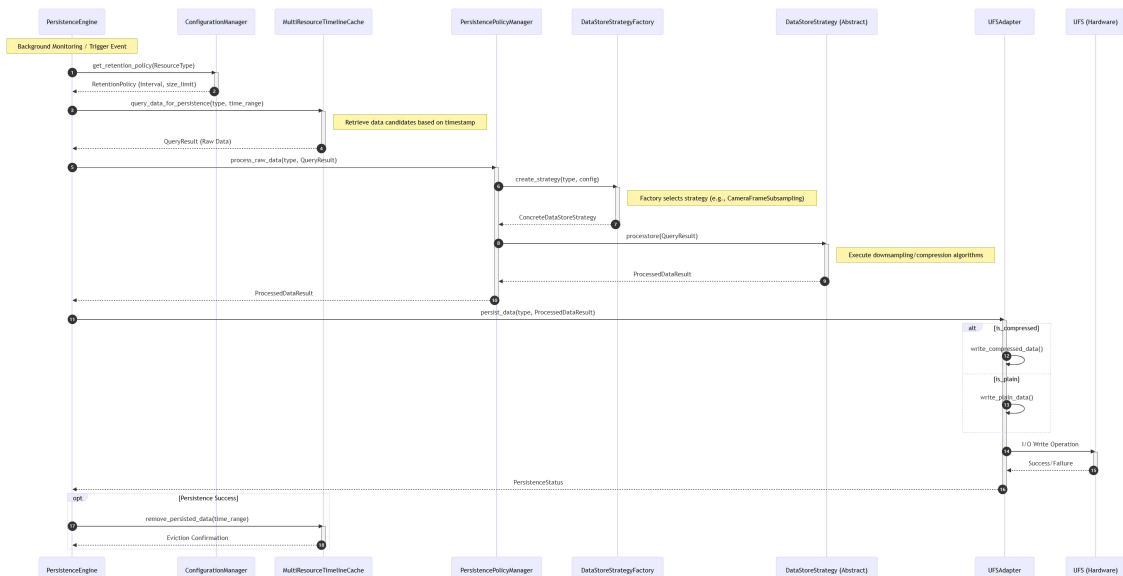


图 6 持久化模块时序图

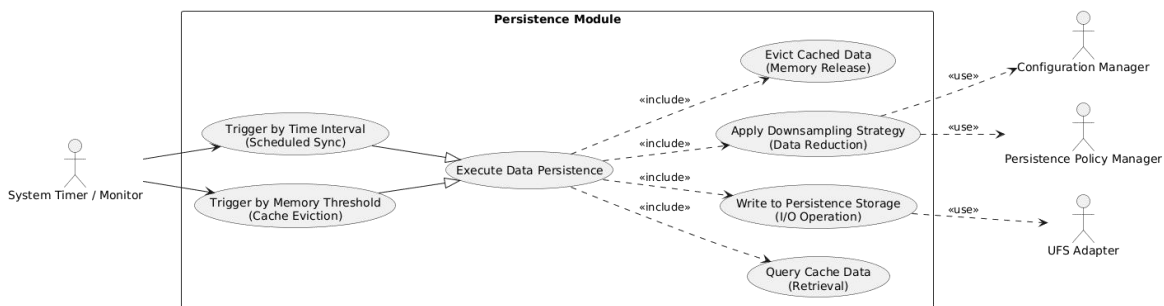


图 7 持久化模块用例图

2.2.4. 支持触发器功能，当数据的预定状态转换规则发生时，可以事件通知智能体层

针对智能座舱场景下的主动感知与关键事件回溯需求，系统将构建一套灵活的事件触发器与时间线检查点机制（如图 8 与图 9）。在触发规则配置方面，引擎需具备对多模态数据的细粒度监控能力，具体支持针对车辆信号（Vehicle Signal）定义单一信号或信号组的特定状态跃迁、阈值触发规则，以及针对特定资源捕获事件的实时侦测逻辑，以此实现数据驱动的业务唤醒。同时，为强化时间线数据的语义索引能力，系统应支持多元化的检查点（Checkpoint）生成策略，既允许上层智能体（Agent）根据推理结果主动注入业务级检查点，也支持通过预设触发器规则在检测到关键事件时自动在时间线上锚定检查点，从而为后续的数据查询与持久化精简提供精确的时空标记。

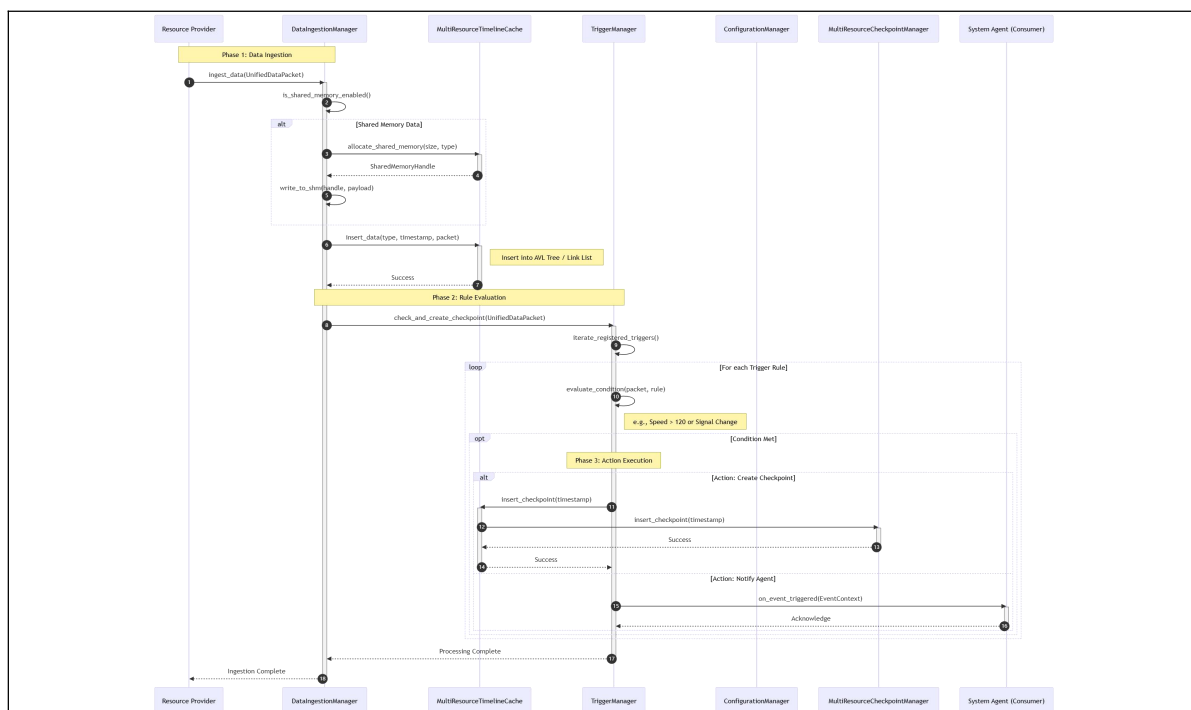


图 8 触发器模块时序图

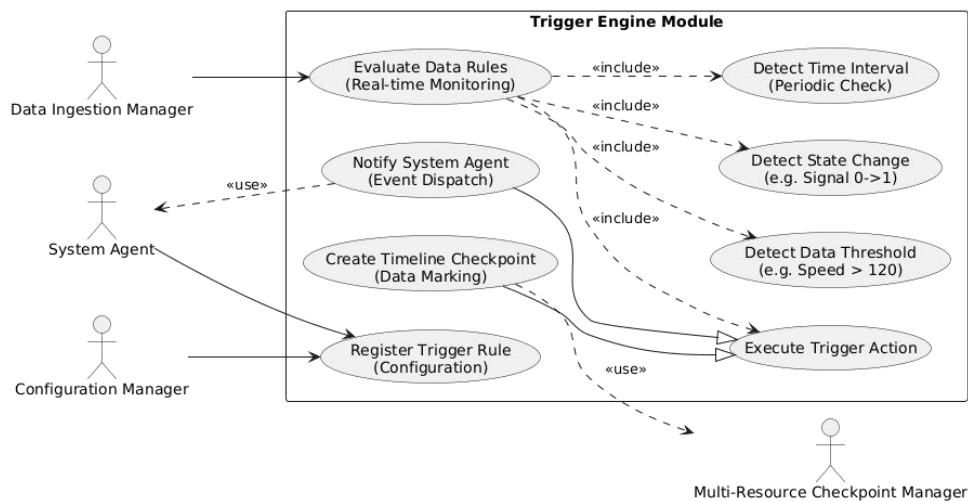


图 9 触发器模块用例图

2.3. 非功能性需求

资源数据类型可扩展。

资源数量为 5，Vehicle 信号数量不超过 100 的条件下，缓存中全类型资源查询和单资源查询响应时间不超过 10ms，UFS 中不超过 100ms。

资源数量为 5, Vehicle 信号数量不超过 100 的条件下, 单个文本和信号资源写入不超过 1ms, 单个复杂存储资源写入不超过 5ms。

3. 系统概要设计

3.1. 系统架构设计

本系统基于“时间线中心化（Timeline-Centric）”与“分层解耦（Layering&Decoupling）”的设计理念，构建了一个面向端侧大模型的高性能多模态资源管理引擎。系统旨在解决异构数据源（如车辆信号、音视频流、系统日志）的统一接入、时序对齐、高效缓存与智能分发问题。

如图 10 所示，系统整体架构在逻辑上划分为四个核心层级：控制平面（Control Plane）、服务接口层（Service Interface Layer）、核心引擎层（Core Engine Layer）以及资源适配层（Resource Adaptation Layer）。各层级之间通过明确定义的接口契约进行交互，确保了系统的高内聚与低耦合。此外，系统引入配置管理与持久化策略模块作为辅助支撑，以满足不同业务场景下的动态配置与数据生命周期管理需求。

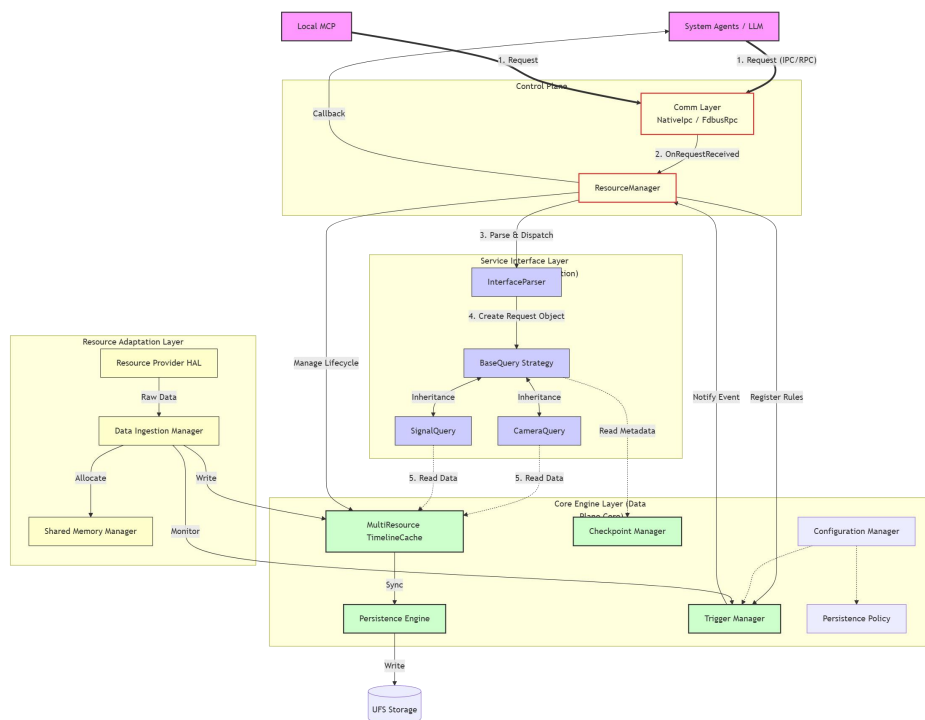


图 10 系统架构图

3.2. 软件结构与模块抽象

为了支撑上述复杂的多模态数据处理流程，系统采用了模块化与分层解耦的软件设计思想。如图 11 所示，系统内部逻辑被抽象为数据缓存、持久化管理、查询处理、数据摄取及配置管理等核心功能域，各功能域通过定义良好的接口进行交互，实现了高内聚低耦合的架构特性（如图 11）。

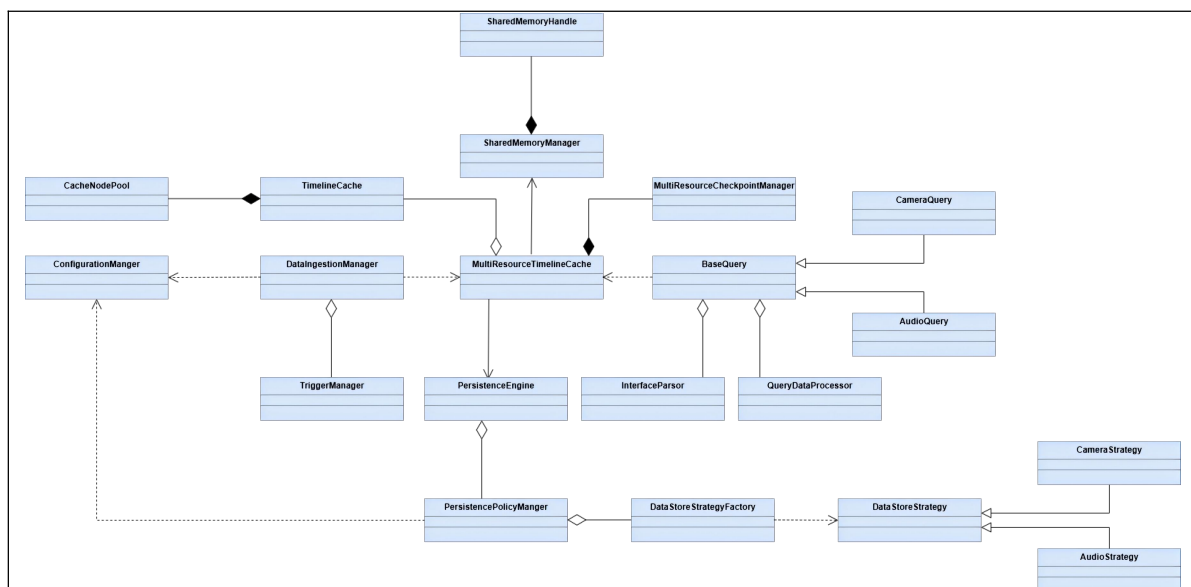


图 11 资源层类图

针对高清视频流（Camera Frames）与屏幕截图（Screen Capture）等大载荷数据，传统的基于 Socket 的 IPC 通信会因内核态与用户态之间的数据拷贝产生显著延迟。本课题设计了基于引用计数的共享内存管理机制。系统在初始化阶段预分配一块连续的物理内存池，并将其映射到 Resource Provider 与 Engine 的虚拟地址空间。当 Provider 产生视频帧时，直接向内存池申请 Block 并写入数据，随后仅将包含内存偏移量、大小及引用计数对象的句柄通过 IPC 通道传递给 Timeline Cache。Engine 接收句柄后，仅增加引用计数而不动物理数据。当且仅当引用计数归零（即无任何 Agent 引用且已完成持久化）时，内存块才会被回收。通过引用计数机制，实现了大容量数据在跨进程传输中的零拷贝（Zero-Copy）管理，有效降低了内存开销。

此外为了解耦存储逻辑与具体的业务规则，系统引入了策略模式。PersistenceEngine 作为持久化引擎，不直接硬编码存储逻辑，而是依赖 PersistencePolicyManager 和 DataStoreStrategyFactory。针对不同的资源类型（如 Camera 或 Audio），工厂类会实例化具体的策略类（如 CameraFrameSubsamplingStrategy 或 AudioDownsamplingStrategy）。这种设计使得系统能够根据配置灵活地应用抽帧、降采样或压缩算法，而无需修改核心代码。

在查询侧，BaseQuery 定义了标准化的查询模版，而 InterfaceParser 则充当适配器角色，负责将来自 IPC、RPC、SQL 等不同协议的外部请求归一化为系统内部的标准查询对象 NativeQueryRequest。这种设计确保了系统能够无缝对接 System Agents、Local MCP 等多种上层应用，同时保持了内核逻辑的稳定性。

3.3. 数据生命周期与动态行为

为了确保海量多模态数据在有限的嵌入式资源中高效流转，系统建立了严格的数据生命周

期管理机制。如图 12(数据生命周期状态机视图)所示，一个标准的数据单元(UnifiedDataPacket)从进入系统到最终消亡，其状态流转过程被设计为确定的有限状态机(如图 12)。

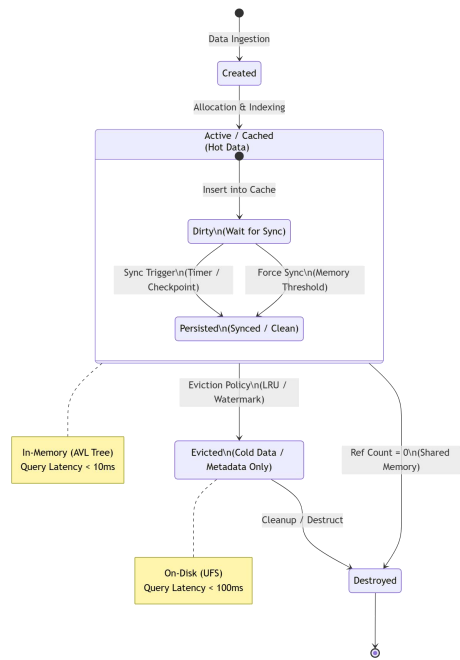


图 12 数据生命周期状态机图

该动态模型主要包含三个关键阶段的管控，首先是数据注入：数据经由 `DataIngestionManager` 接入后，处于暂态，此时系统完成时间戳对齐与格式标准化。对于视频流等大容量载荷，系统通过 `SharedMemoryManager` 分配共享内存句柄，数据直接进入共享驻留态，通过引用计数机制管理其生命周期，避免了内核态与用户态之间的内存拷贝开销。随后，数据节点被挂载至 `TimelineCache` 的数据结构中，进入活跃态（Active/Hot），对外提供 $O(\log N)$ 复杂度的实时查询服务。

随着时间推移或缓存容量达到阈值，`PersistencePolicyManager` 会根据预设策略（如 LRU 或时间窗口策略）标记部分活跃数据进入待持久化态。此时，`PersistenceEngine` 异步介入，调用相应的精简策略（如 `CameraFrameSubsampling`）对数据进行降采样或压缩处理，并将其写入 UFS 存储。一旦写入成功并更新索引文件，数据即转变为持久化态（Persisted/Cold），这一过程实现了冷热数据的平滑交换。

当内存缓存空间不足时，已完成持久化的数据节点将从内存中移除，进入驱逐态（Evicted）。对于共享内存资源，系统会监测其引用计数，仅当计数归零（即无任何 Agent 或缓存节点引用该数据块）时，才会触发底层的内存释放操作，从而彻底终结该数据的生命周期。

通过这种状态机驱动的管理模式，系统不仅保证了数据在并发环境下的状态一致性，还有效防止了内存泄漏与悬挂指针问题，确保了 7x24 小时运行的稳定性。

3.4. 混合通信架构设计

本系统架构在“控制面与数据面分离”的基础上，构建了混合通信子系统（如图 13）：

数据面：由 TimelineCache 和 SharedMemoryManager 构成。它专注于数据的内存索引、存储计算和零拷贝传输。数据面对应具体的物理内存操作，对上层的通信协议完全透明。控制面：由 ResourceManager 主导。它不直接处理底层 Socket 或 FDBus 细节，而是通过策略模式管理通信接口，负责指令的路由、权限校验和生命周期管理。

为了屏蔽底层传输协议（Unix Domain Socket 与 FDBus）的差异，系统定义了统一的服务器抽象接口 IServer。IServer 是所有通信服务的基类。它定义了服务器的生命周期（启动/停止）、请求回调注册以及统一的回包接口。无论底层是基于文件描述符的 Socket 写入，还是基于中间件的消息回复，都在此层被抽象为 send_response。

系统实现了两种具体的通信策略：

Native IPC(本地通信)，基于 Unix Domain Socket(如/tmp/engine.sock)。适用于同一计算单元内的 Agent。控制指令（如 Query 请求）通过 Socket 发送，而大数据载荷（如视频帧）仅传输 SharedMemoryHandle，从而实现零拷贝，确保查询响应时间满足小于 10ms 的指标。

FDBus RPC(远程通信)，基于 FDBus 中间件，注册服务名（如 org.bosch.timeline.engine）。适用于跨域（如 IVI 与 ADAS 之间）交互。它将系统的统一协议封装在 FDBus 的 payload 中传输。虽然延迟略高于本地 IPC，但提供了跨节点的网络透明访问能力。

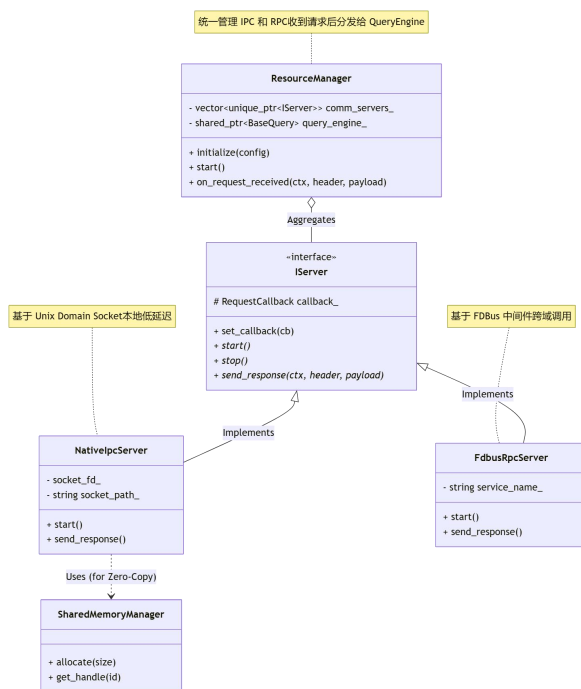


图 13 通信类图

3.5. 并发数据处理与实时性保障机制

针对智能座舱场景下多模态数据不仅流量大（如多路高清视频流），且具有突发性高频写入（如瞬时超过 100 路 CAN 信号并发）的特征，传统的单体锁机制与同步 I/O 模型难以满足系统对写入延迟（ $<1\text{ms}$ ）和查询响应（ $<10\text{ms}$ ）的严苛要求。为此，本课题拟设计一套基于混合线程模型与细粒度并发控制的高性能处理架构，以确保系统在重负载下写入的确定性低延迟（如图 14）。

(1) 基于 Reactor 模式的混合线程架构设计

为了解决 I/O 密集型任务与计算密集型任务的资源争抢问题，本系统将摒弃传统的“每个连接一线程”（Thread-per-connection）模型，转而采用 I/O 线程与工作线程分离的 Reactor 模式。

数据接入层采用轻量级 I/O 多路复用技术（在 Linux 环境下使用 `epoll` 机制）构建单 I/O 线程（IO-Reactor）。该线程仅负责监听文件描述符（FD）的读写事件、处理连接建立与原始二进制流的接收，不涉及任何具体的协议解析或内存分配操作。这种设计能够以极低的上下文切换开销应对数百路传感器信号的同时接入。

后端维护一个固定大小的工作线程池。I/O 线程在接收到完整数据包后，将其封装为任务分发至工作线程。工作线程负责执行 `UnifiedDataPacket` 的解码、时间戳对齐、共享内存分配（针对大容量数据）以及 `TimelineCache` 索引更新等耗时操作。通过这种分层架构，系统能够有效隔离 I/O 抖动对核心业务逻辑的影响，确保高频信号的吞吐能力。

(2) 基于无锁环形队列的零拷贝缓冲机制

在 I/O 线程与工作线程的数据交互环节，为避免互斥锁带来的线程阻塞与上下文切换开销，本课题拟引入单生产者-单消费者无锁环形队列作为数据注入的缓冲区。

无锁设计：利用 C++11 标准中的 `std::atomic` 原子操作与内存屏障技术，管理队列的头尾指针，实现数据的无锁入队与出队。这保证了在中断上下文或高频信号到达时，数据接入层能够以纳秒级的延迟完成数据暂存，彻底消除了锁竞争导致的性能波峰。

流量削峰与零拷贝：环形队列不仅作为流量整形缓冲区吸收瞬时突发流量，防止数据丢失；同时结合共享内存句柄机制，在队列中仅传递数据的元数据指针而非载荷本身，实现了数据从接入到处理全流程的“零拷贝”流转，满足视频流等大数据量资源的低延迟写入需求。

(3) 多粒度分段锁并发控制策略

针对核心数据结构 `TimelineCache` 在高并发读写场景下的数据一致性保护，本系统将放弃全局大锁，采用分段锁与读写锁相结合的细粒度控制策略。

`MultiResourceTimelineCache` 内部维护一个基于 `ResourceType` 的哈希映射表，对 `Camera`、`Audio`、`VehicleSignal` 等不同资源类型的缓存实例使用独立的互斥锁。这确保了高频的 CAN 信号

写入操作绝不会阻塞对摄像头视频流的查询请求，实现了不同模态数据的并发隔离。

为了满足查询响应的实时性需求，查询路径摒弃了写入侧的缓冲队列机制，采用直接访问存储层的旁路设计。系统根据数据热度与处理复杂度将查询请求动态分流为“快路径”与“慢路径”两种处理模式。

针对内存驻留的热点数据查询且无需复杂后处理的场景，系统启用基于共享锁的同步直读快路径。在此模式下，RPC 服务线程在当前上下文直接解析请求，并申请 **TimelineCache** 的共享读锁进行访问。得益于系统的分段锁设计，读写锁的竞争仅局限于当前活跃的时间桶，而历史时间桶处于不可变状态，实现了近乎无锁的高效访问。该路径有效避免了线程切换开销，可实现微秒级的确定性响应，并通过直接返回指向内存数据块的共享内存句柄，实现了全链路零拷贝。

相对地，针对持久化存储的冷数据回溯或需经由 **QueryDataProcessor** 执行高负载后处理的场景，系统采用基于 **Worker Pool** 的异步任务卸载慢路径。为防止耗时操作阻塞核心通信线程，RPC 线程将请求封装为异步任务，提交至独立的查询工作线程池，随即释放当前资源以响应其他高优先级请求，有效规避了队头阻塞问题。在处理流程中，工作线程利用 **mmap** 技术建立文件映射，在避免物理 I/O 阻塞的同时完成数据的解压与格式化计算，并将处理后的业务数据写入一块新分配的临时共享内存。任务完成后通过回调机制通知 **RPC** 层，返回包含处理结果的新句柄，从而在保障主线程吞吐量的同时实现了复杂计算结果的高效流转。

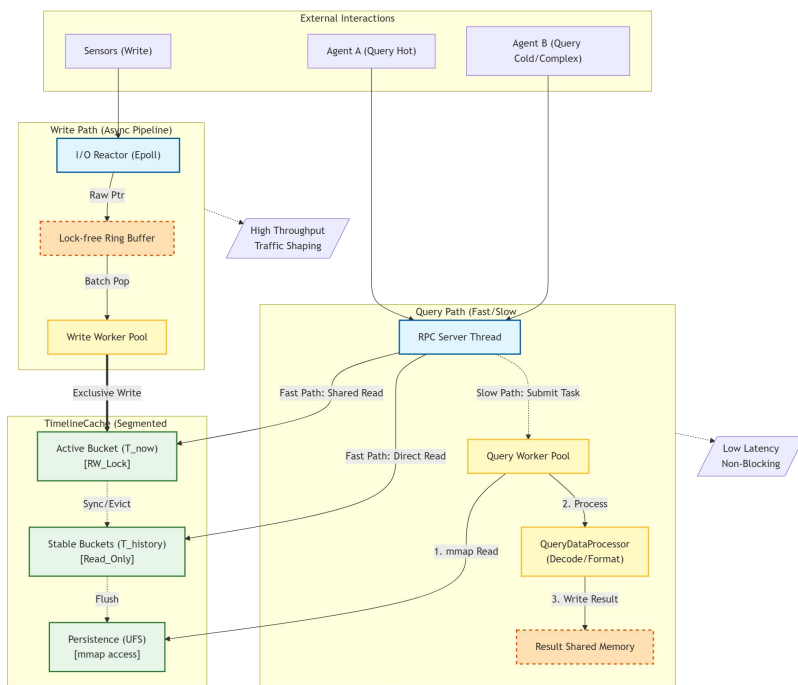


图 14 并发类图

4. 实验验证方案

本课题将搭建基于高通 QC8397/CX1 车载计算平台的硬件环境，从以下三个维度对资源引擎进行验证：

基准性能测试：编写压力测试脚本，模拟 100 路 CAN 信号(100Hz)与 4 路高清视频流(30fps)的并发注入。通过打点统计 ingest_data 接口的延迟，验证是否满足<1ms（信号）和<5ms（视频）的设计指标。以及在 Cache 填充率达到 80%的工况下，并发发起 50 个 Agent 的范围查询请求，监控 query_by_range 接口的平均响应时间与 CPU 占用率。

持久化策略有效性验证：对比开启与关闭“检查点压缩策略”下的磁盘占用空间增长曲线，计算数据压缩比，验证降采样算法在长周期运行下的存储优化效果。验证在 UFS 高负载读写背景下，前台 Cache 查询是否存在阻塞现象，以此评估异步持久化线程模型的有效性。

系统集成与稳定性测试：构建典型的“疲劳驾驶监测”场景（如图 15）：配置 Trigger 规则监测驾驶员闭眼信号。当触发闭眼事件时，验证系统是否能自动生成 Checkpoint，并正确回溯并锁定事件发生前 10 秒的视频数据不被覆盖，确保关键证据链的完整性。

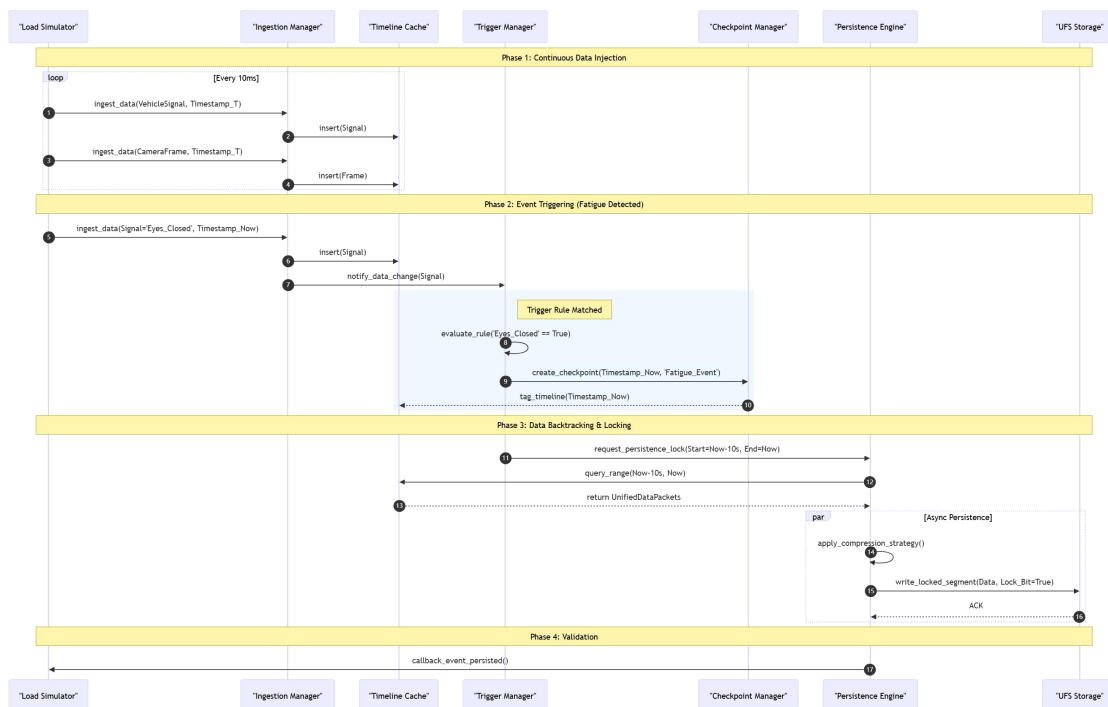


图 15 疲劳驾驶检测时序图

5. 技术难点及解决方案

针对高并发场景下异构时间序列数据的索引构建与内存管理问题，系统需在毫秒级延迟约束下处理车辆信号、视频流及异步事件的混合写入，并维持时间戳索引的有序性。为此，项目

摒弃了传统的线性表存储方式，设计了基于红黑树的 TimelineCache 结构，确保数据插入与检索的时间复杂度稳定在 $O(\log N)$ 。同时，为解决节点频繁创建与销毁引发的内存碎片化问题，引入对象池技术，通过设计 CacheNodePool 模块预分配内存块，有效降低了 malloc/free 系统调用的频率，显著提升了缓存命中率与内存利用效率。

对于在大吞吐量多模态数据的传输与生命周期管理，为了避免视频与音频数据在 Resource Provider、Engine 与 Agent 间进行深拷贝而导致的 CPU 中断与总线拥塞，项目确立了控制流与数据流分离的架构设计。系统利用操作系统级共享内存作为物理载体，并通过 SharedMemoryManager 模块统一管理内存句柄。TimelineCache 的索引节点仅存储指向共享内存块的轻量级指针及元数据，配合引用计数机制管理资源生命周期。该机制确保了当所有消费者释放引用后，底层共享内存即被安全回收，从而实现了全链路的零拷贝传输。

在混合通信协议的统一适配与透明化调用方面，系统需兼容本地进程间通信、跨域远程过程调用及 SQL 查询接口。针对协议封包格式差异导致的业务逻辑耦合问题，项目构建了通信抽象层，定义 IServer 与 IConnection 抽象接口，将底层 Socket 缓冲区、共享内存句柄与 FDBus 消息封装为统一的标准上下文。基于策略模式分别实现 NativeIpcServer 与 FdbusRpcServer，通过依赖注入在运行时动态绑定通信策略。此外，设计 InterfaceParser 模块采用递归下降算法解析请求，将其归一化为统一的 Query 对象，实现了业务逻辑对底层传输协议的完全透明。

针对基于数据时效性的异步持久化问题，为了解决有限内存资源下的数据换页与海量历史数据存储膨胀矛盾，项目设计了基于双缓冲队列的异步持久化机制 PersistenceEngine，将数据落盘操作剥离至独立后台线程以避免阻塞主线程。

四、预期成果

- （一）完成多模态实时数据引擎的设计与实现
- （二）撰写工程硕士学位论文

五、已取得的与学位申请成果研究内容相关的成果

暂无

六、研究计划与进度安排

2025.11~2025.12 对多模态数据融合、时序数据缓存技术及嵌入式存储方案进行调研，分析国内外在座舱数据管理领域的现状，初步设计基于时间线的资源引擎实施方案。

2025.12~2026.01 确定 AI 座舱场景下的功能与性能需求，完成 TimelineCache、PersistenceEngine 等核心模块的结构设计与接口定义。

2026.01~2026.04 开发数据摄入、内存缓存管理、持久化存储及查询接口四个核心功能模块，并进行单元测试、模块联调并修正逻辑错误，完成阶段性技术文档与开题报告的修订。

2026.04~2026.07 完成查询引擎与数据后处理模块的系统集成，并部署至 QC8397/CX1 上，进行全链路压力测试与集成测试。

2026.07~2026.09 优化 Cacheline 与共享内存管理代码，对系统进行详细的性能分析与缺陷修复；同时整理实验数据，完善设计文档，撰写毕业论文，准备毕业答辩。

研究生本人签名：王鹏焜

校内导师（主导师）签名：王鹏焜

实践导师签名：王鹏焜

2025年12月20日